

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

Отчет по лабораторной работе №2
по дисциплине «Системы искусственного интеллекта»

Выполнил:
студент группы ИСТбд-31 Романов И.Н

Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

Цель:

Разработать алгоритм решающий задачу о разбиении городов на K кластеров 3 разными способами и сравнить результаты работы.

Алгоритмический:

1. Назначение первого центра: Программа берет самый первый город из списка и назначает его первым центроидом.
2. Поиск самого удаленного города: Программа смотрит на все остальные города и ищет тот, который находится дальше всего от первого. Этот город становится вторым центроидом.
3. Заполнение до K центроидов : Для каждого оставшегося города рассчитывается расстояние до ближайшего к нему уже существующего центроида. Из всех этих «минимальных расстояний» выбирается самое максимальное.
4. Распределение : Когда все K центроидов выбраны, каждый город просто приписывается к тому центру, который к нему ближе.

Ручной k-means:

1. Назначение первых центроидов: Программа выбирает K случайных точек из общего списка городов и объявляет их начальными центроидами.
2. Распределение: Для каждого из городов рассчитывается расстояние до всех K центроидов. Город «записывается» в кластер к тому центру, который оказался к нему ближе всего.
3. Назначение новых центроидов: Когда все города распределены, старые центроиды удаляются. Программа вычисляет новые координаты для каждого кластера как среднее арифметическое координат всех городов, попавших в эту группу
4. Стабилизация : Шаги 2 и 3 повторяются до тех пор, пока координаты центроидов не перестанут меняться. Это означает, что алгоритм нашел оптимальное положение для разделения городов.

ИИ (k-means):

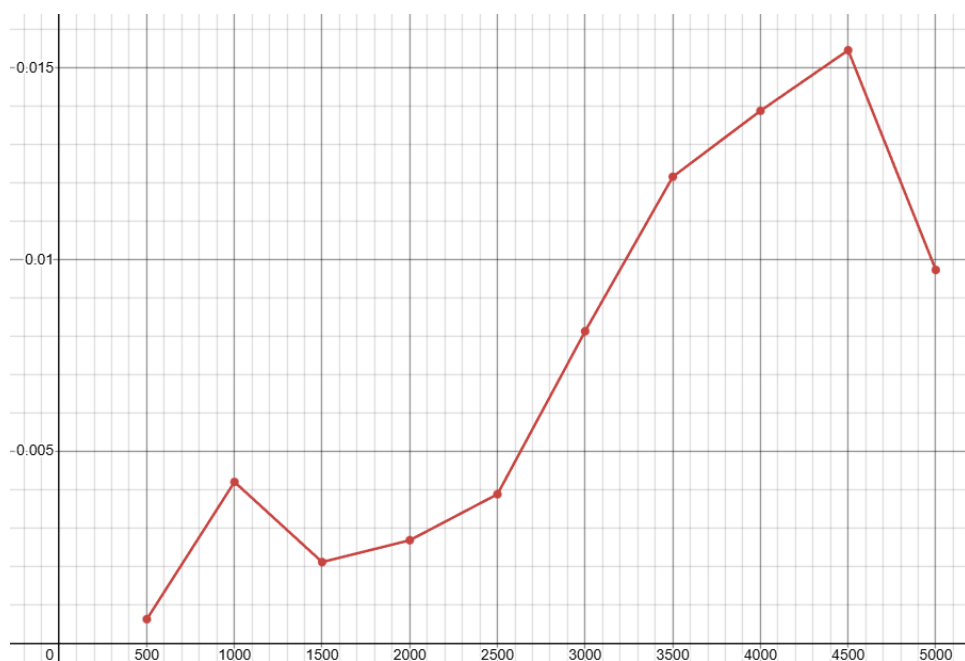
k -means алгоритм кластеризации без учителя (unsupervised learning), который разбивает набор данных на k непересекающихся групп (кластеров). Цель сделать объекты внутри одного кластера максимально похожими друг на друга, а между кластерами — максимально разными.

Графики работы алгоритмов по времени:

Кол-во городов: от 500 до 5000

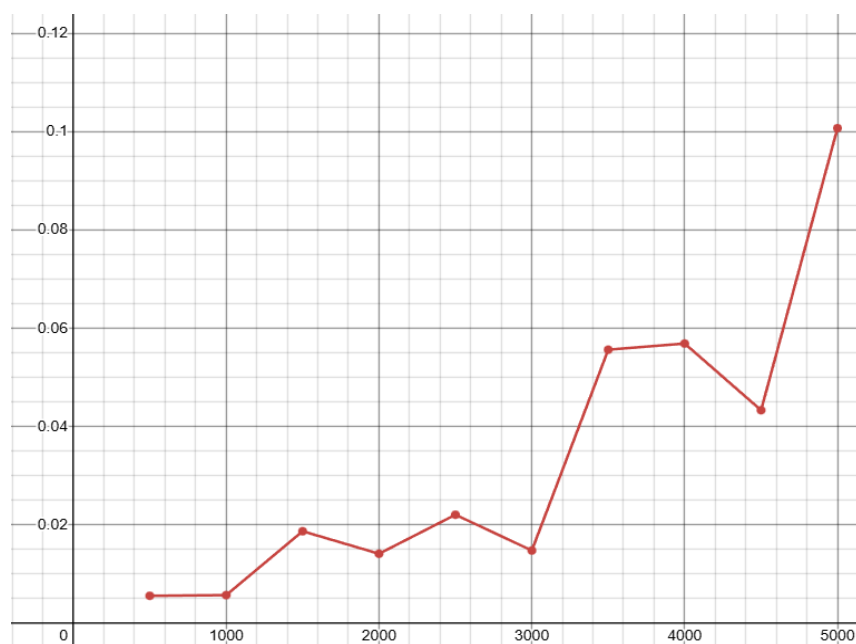
Алгоритмический:

x_1	 y_1
500	0.00063
1000	0.00421
1500	0.00212
2000	0.00269
2500	0.00389
3000	0.00814
3500	0.01217
4000	0.01389
4500	0.01547
5000	0.00974



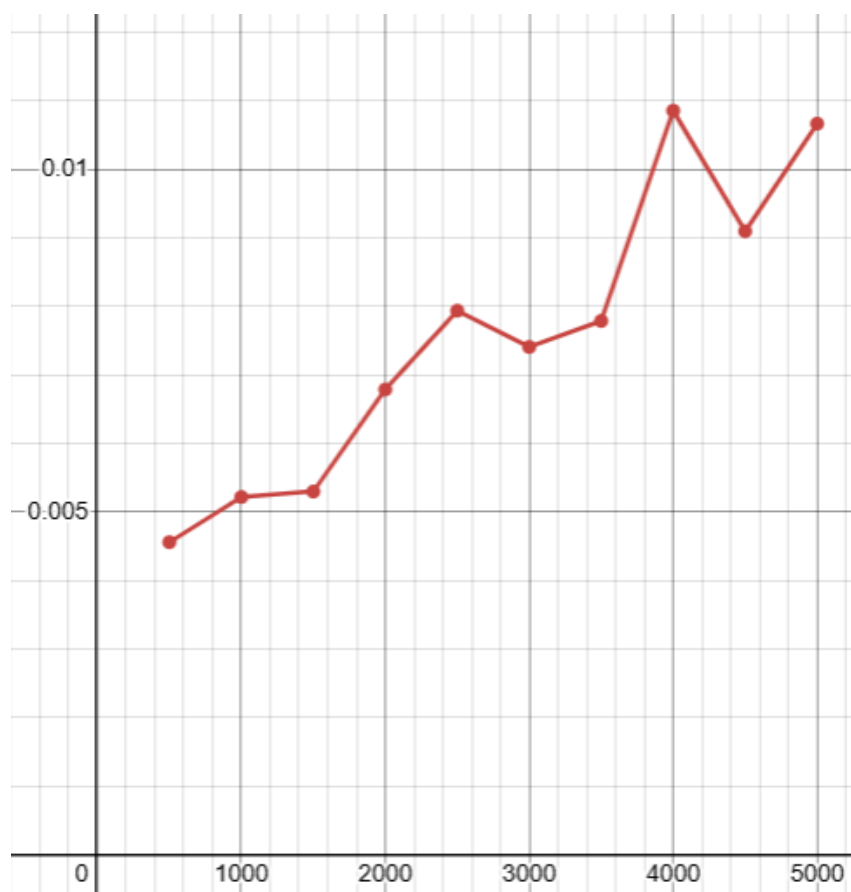
Ручной k-means:

x_1	 y_1
500	0.00560
1000	0.00574
1500	0.01874
2000	0.01416
2500	0.02210
3000	0.01482
3500	0.05569
4000	0.05694
4500	0.04337
5000	0.10077



ИИ (k-means):

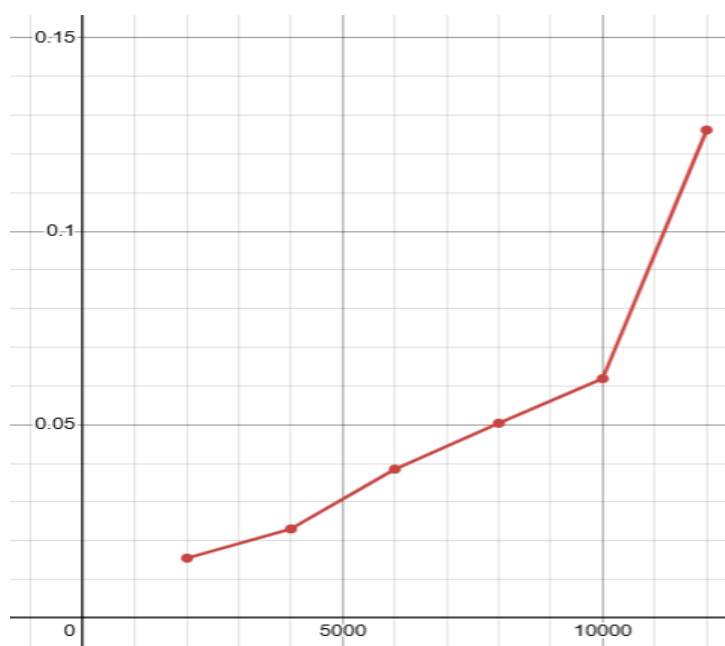
x_1	y_1
500	0.00456
1000	0.00522
1500	0.00530
2000	0.00679
2500	0.00794
3000	0.00741
3500	0.00779
4000	0.01086
4500	0.00910
5000	0.01067



Кол-во городов: от 2000 до 12000

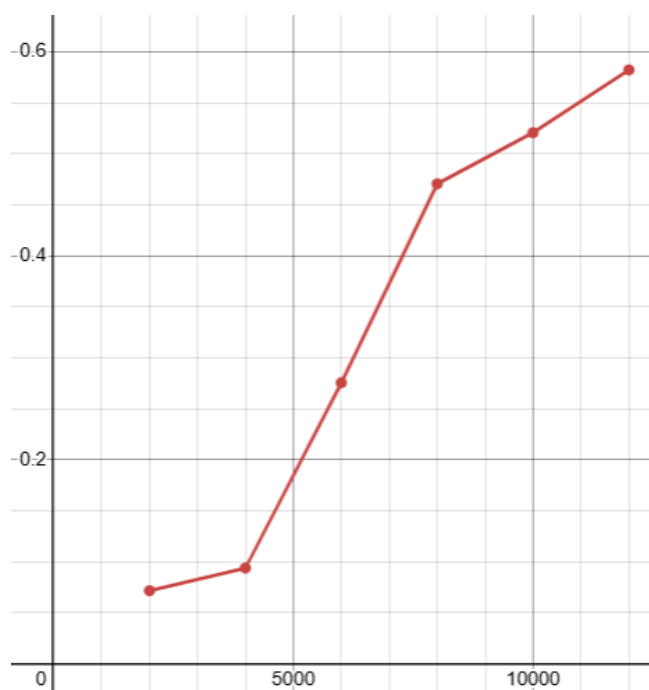
Алгоритмический:

x_1	y_1
2000	0.01554
4000	0.02310
6000	0.03855
8000	0.05041
10000	0.06195
12000	0.12620



Ручной k-means:

x_1	 y_1
2000	0.07162
4000	0.09382
6000	0.27556
8000	0.47061
10000	0.52081
12000	0.58238



III (k-means):

x_1	 y_1
2000	0.01474
4000	0.01558
6000	0.01607
8000	0.02140
10000	0.02347
12000	0.02482

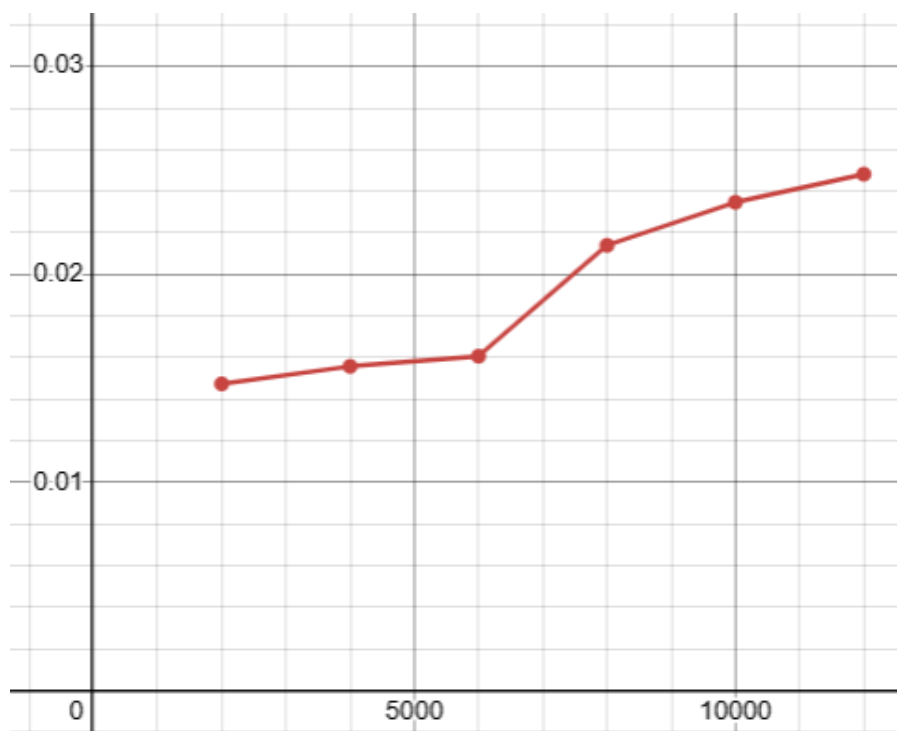


Таблица качества разбиения данных

Города	Алгоритмический	Ручной k-means	ИИ
500	43695008	20403127.31	20403127.31
1000	107205260.0	40489554.37	40491101.11
1500	117548743.0	53050349.81	53048322.12
2000	193792542.0	82710827.74	82711473.55
2500	281878720.0	104789778.67	104804701.21
3000	354067703.0	123902211.28	123898640.68
3500	352584515.0	147839107.83	147880664.27
4000	293592672.0	138399831.86	137144835.53
4500	493912651.0	189596447.39	189600960.12
5000	337087533.0	147448983.11	147630271.65

Вывод: При малом числе городов алгоритмический метод выигрывает по скорости, но проигрывает ИИ по качеству кластеризации: WCSS у ИИ оказывается ниже. С ростом объёма данных ситуация меняется — ИИ превосходит алгоритм и по времени и по WCSS.

Листинг кода:

```
import math
import random
import time
from sklearn.cluster import KMeans

#k = int(input("Введите кол-во кластеров:"))

file = open("fa.txt", "r", encoding="utf8")

# Создание массива с данными [город, x, y] и массива [x, y]
b = []
b1 = []
for i in file:
    i1 = i.strip().split()
    b.append([i1[0], float(i1[1]), float(i1[2])])
    b1.append([float(i1[1]), float(i1[2])])

# Создание матрицы расстояний
n = len(b)
dist = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(i + 1, n):
        c = math.sqrt((b[i][1] - b[j][1]) ** 2 + (b[i][2] - b[j][2]) ** 2)
        dist[i][j] = c
        dist[j][i] = c

#Метод Локтя
def elbow_method(b):
    all_wcss = []
    k = 1

    for i in range(1, 15):
        clusters, centroids = kmeans1(i, b, True)
        clwcss = calcwcss(clusters, centroids, b)
        all_wcss.append(clwcss)

    for i in range(1, len(all_wcss)):
        s = ((all_wcss[i-1] - all_wcss[i])/all_wcss[i-1])*100
        if s < 15:
            k = i
            break
    return k

# Алгоритмический
def algorithm(k, b, dist, wcss=False):
    start = time.perf_counter()

    n = len(b)
    centroids = [0]

    # Выбор центроидов
    while len(centroids) < k:
        maxdist = 0
```

```

city = 0
for i in range(n):
    if i in centroids: continue
    dista = min(dist[i][c] for c in centroids)
    if dista > maxdist:
        maxdist = dista
        city = i
centroids.append(city)

clustres = {}
clustersid = {}
for i in centroids:
    clustres[i] = []
    clustersid[i] = []

# Заполнение кластеров по принципу отнесения каждого города к ближайшему центроиду
for i in range(n):
    city_name = b[i][0]

    min_dist = float("inf")
    namecit = 0

    for ceni in centroids:
        cendist = dist[i][ceni]

        if cendist < min_dist:
            min_dist = cendist
            namecit = ceni

    clustres[namecit].append(city_name)
    clustersid[namecit].append(i)
end = time.perf_counter()

if wcss == True:
    centroidscord = []
    clusters_id = {}

    for i in range(len(centroids)):
        ka = centroids[i]
        centroidscord.append([b[ka][1], b[ka][2]])
        clusters_id[i] = clustersid[ka]

    return clusters_id, centroidscord

else:
    #print('-----Алгоритмический-----')
    print(f"Время 'Алгоритмический': {end-start:.5f} сек")

    #print("\nКластеры:")
    #for i in range(1, k + 1):
    #    print(f"Кластер {i}: {clustres[centroids[i-1]]}")

# KMeans
def kmeans(k, b1, b, wcss=False):
    start = time.perf_counter()

    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(b1)
    centroids = kmeans.cluster_centers_.tolist()

    end = time.perf_counter()

```



```

# 3. Формируем словарь кластеров с названиями городов и кластер с индексами
clustres = {}
clustersid = {}
for i in range(k):
    clustres[i] = []
    clustersid[i] = []

# Индексы городов по кластерам
for i in range(len(labels)):
    idy = labels[i]
    clustersid[idy].append(i)

# Названия городов по кластерам
for i in range(len(b)):
    cluster_id = labels[i]
    city_name = b[i][0]
    clustres[cluster_id].append(city_name)

if wcss == True:
    return clustersid, centroids
else:

    #print("-----KMeans-----")
    print(f"Время 'KMeans': {end-start:.5f} сек")

    #print("\nКластеры:")
    #for c in clustres:
        #print(f"Кластер {c+1}: {clustres[c]}")

# Ручной KMeans
def kmeans1(k, b, wcss=False):
    start = time.perf_counter()

    n = len(b)
    centroidsid = random.sample(range(n), k)
    centroids = []
    for i in centroidsid:
        centroids.append([b[i][1], b[i][2]])

    while True:

        clusters = {}
        for i in range(k):
            clusters[i] = []

        # Заполнение кластеров по принципу отнесения каждого города к ближайшему центроиду
        for i in range(n):
            city_x = b[i][1]
            city_y = b[i][2]
            min_dist = float("inf")
            icit = 0

            for cen in range(k):
                cen_x = centroids[cen][0]
                cen_y = centroids[cen][1]
                dista = math.sqrt((city_x - cen_x) ** 2 + (city_y - cen_y) ** 2)

                if dista < min_dist:
                    min_dist = dista
                    icit = cen

            clusters[icit].append(i)

```

```

new_centroids = []
# Новые центроиды
for key in range(k):
    items = clusters[key]
    sum_x = 0
    sum_y = 0

    for item in items:
        sum_x += b[item][1]
        sum_y += b[item][2]

    mid_x = sum_x / len(items)
    mid_y = sum_y / len(items)

    new_centroids.append([mid_x, mid_y])

if centroids == new_centroids:
    break

centroids = new_centroids

name_clusters = {}
for key, items in clusters.items():
    name_list = []
    for i in items:
        name = b[i][0]
        name_list.append(name)
    name_clusters[key] = name_list
end = time.perf_counter()

if wcss == True:
    return clusters, centroids
else:
    #print("----Ручной KMeans----")
    print(f"Время 'Ручной KMeans': {end-start:.5f} сек")

    #print("\nКластеры:")
    #for c in name_clusters:
    #    print(f"Кластер {c+1}: {name_clusters[c]}")

#Wcss
def calcwcss(clusters, centroids, b):
    wcss = 0

    for i in range(len(centroids)):
        cen_x = centroids[i][0]
        cen_y = centroids[i][1]

        citys = clusters[i]

        for id in citys:
            city_x = b[id][1]
            city_y = b[id][2]

            dist = (city_x - cen_x)**2 + (city_y - cen_y)**2

            wcss += dist
    return wcss

k = elbow_method(b)

print(f"\n===Статистика (K = {k})===")

```

```
print("---Алгоритмический:")
aa1, bb1 = algorithm(k, b, dist, True)
alg1 = calcwcsc(aa1, bb1, b)
print(f"WCSS: {round(alg1, 2)}")
algorithm(k, b, dist)
```

```
print("\n---Ручной KMeans:")
aa2, bb2 = kmeans1(k, b, True)
alg2 = calcwcsc(aa2, bb2, b)
print(f"WCSS: {round(alg2, 2)}")
kmeans1(2, b)
```

```
print("\n---KMeans:")
aa3, bb3 = kmeans(k, b1, b, True)
alg3 = calcwcsc(aa3, bb3, b)
print(f"WCSS: {round(alg3, 2)}")
kmeans(k, b1, b)
```